



INDIA.RUNS

Build what next India runs on

Team Name : Apex Climbers

Team Leader Name : Pratheeka B

Problem Statement : AI hiring tools score candidates without explaining why, and reward keyword density over real outcomes, we tested this ourselves and found it costs strong candidates the rankings they deserve.

Solution Overview

- **What is your proposed solution?**

IntentRank ranks candidates across five independent signals which is semantic JD alignment, skill depth, behavioral availability, career trajectory, and JD-specific fit, combined into one transparent composite score, instead of a single black-box number.

- **What differentiates your approach from traditional candidate matching systems?**

Two data-integrity passes (honeypot detection + hard filters) run before any scoring, reasoning is generated per candidate from real profile fields only, and outcome evidence is explicitly weighted over keyword density, addressing failure patterns we confirmed by testing Redrob's own existing ranker.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

The JD is decomposed into hard requirements (production-shipped retrieval systems, FastAPI, vector stores), preferred signals (RAGAS, NeMo Guardrails, LangGraph, open source), and implicit expectations (startup-ready, self-directed) before matching begins.

- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Sentence-embedding similarity between weighted candidate text and JD concepts, sustained skill duration and endorsements, recency of platform activity and response rate, and production-vs-research signal words in career descriptions – not literal keyword overlap.

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

A two-stage pipeline: `precompute.py` generates semantic embeddings offline for all candidates, `rank.py` applies four fast heuristic layers, combines all five scores, and outputs the ranked top 100 in under 5 seconds.

- What models, algorithms, or heuristics are used?

sentence-transformers (all-MiniLM-L6-v2) for semantic similarity; rule-based scoring for skill depth, behavioral availability, and career trajectory; zero hosted LLM calls anywhere in the ranking path.

- How are multiple candidate signals combined into a final ranking?

A fixed weighted composite: 35% semantic alignment, 25% skill depth, 20% behavioral availability, 15% career trajectory, 5% JD-specific fit – tuned for this JD's emphasis on production-proven depth over keyword presence.

Explainability & Data Validation

- How are ranking decisions explained?

Every top-100 candidate ships with a reasoning string built entirely from real profile fields like title, company, named skills, response rate, notice period with rank-aware tone and deterministic hash-based variation so reasoning is never templated.

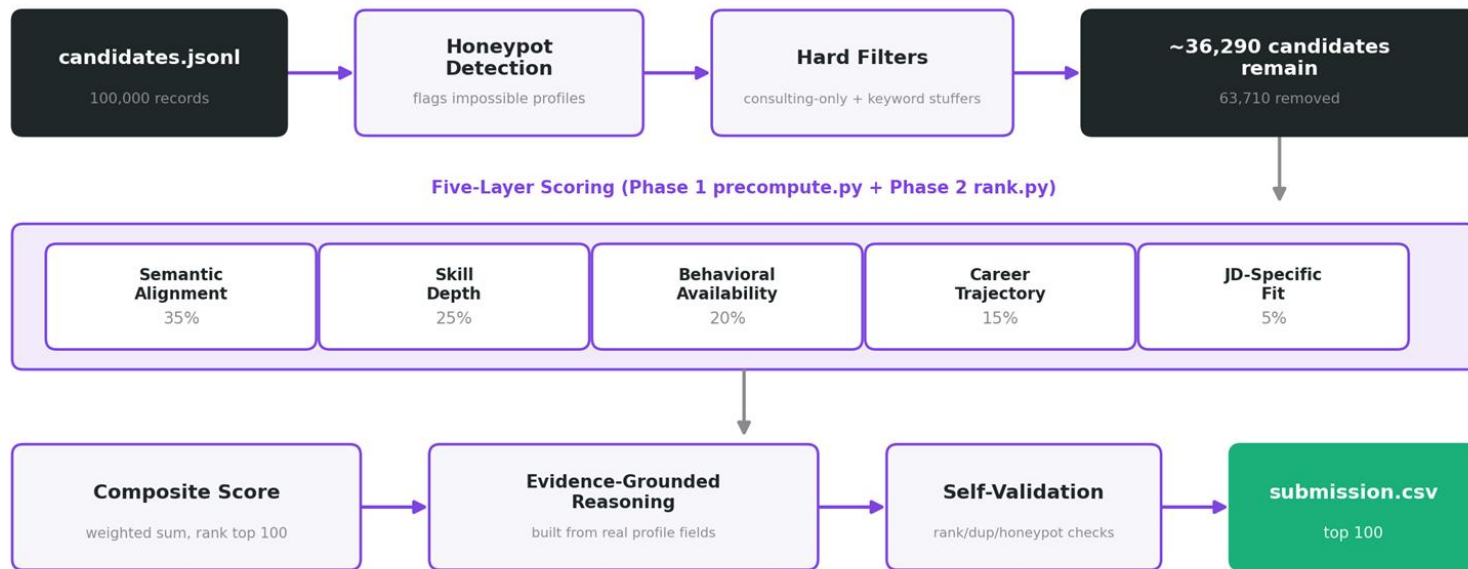
- How do you prevent hallucinations or unsupported justifications?

No claim is generated that cannot be traced to an actual field in the candidate's JSON record; there is no free-form LLM generation step anywhere in the reasoning pipeline.

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

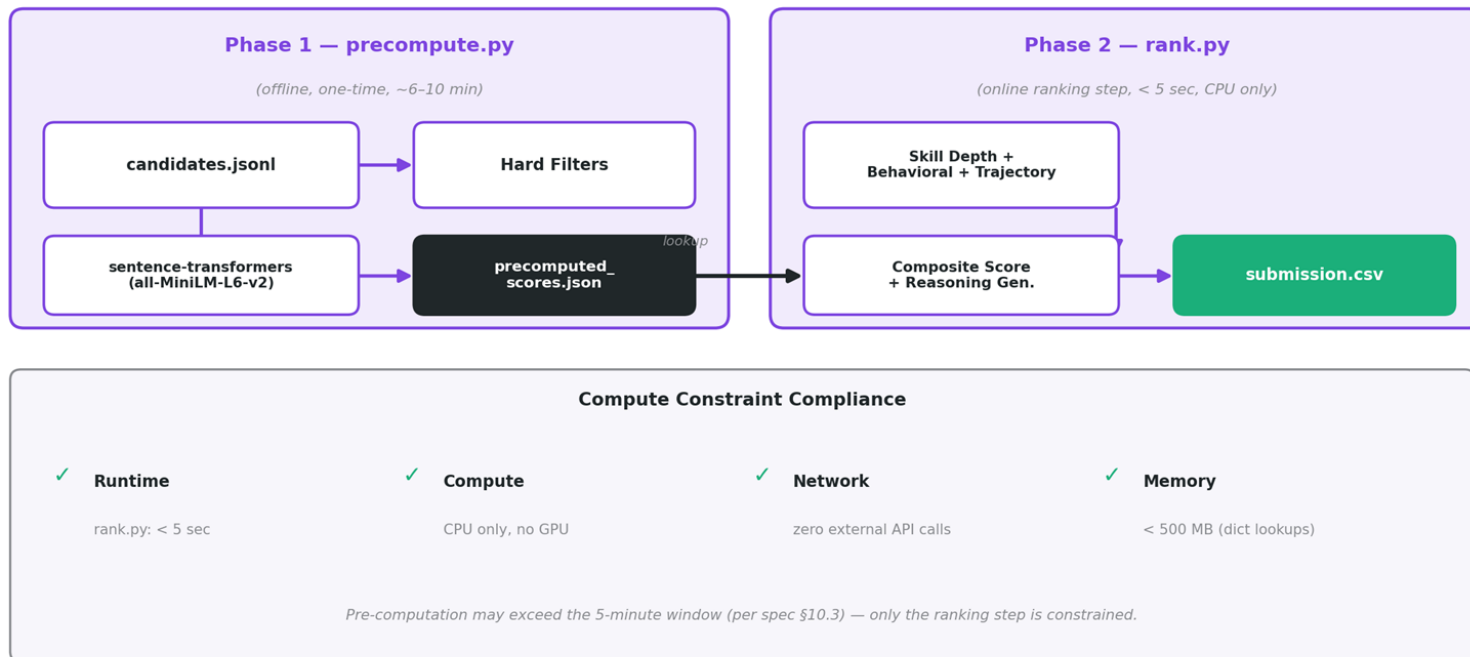
A honeypot detector flags impossible claims (expert skill at 0 months, contradictory career dates) before scoring; verified at 0% honeypot rate in our top-100 output on the full released dataset.

End-to-End Workflow



Verified runtime: precompute ~6-10 min (offline) • rank.py < 5 sec • CPU only • no network calls

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

Hard filters alone removed 63,710 of 100,000 candidates carrying a non-technical title paired with an AI-keyword-stuffed skills list which is confirming the keyword-stuffing failure pattern at full dataset scale.

In our own controlled test of Redrob's existing ranker, a candidate with zero measurable outcomes outranked one with 200+ GitHub stars and shipped libraries; our skill-depth layer is built specifically to correct that.

- How does your solution meet the challenge's runtime and compute constraints?

rank.py completes in under 5 seconds using precomputed embeddings; CPU only, zero GPU dependency, zero external API calls during ranking, under 500MB RAM.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Python - core implementation language for the full two-stage pipeline.

sentence-transformers (all-MiniLM-L6-v2) - lightweight, CPU-friendly semantic embeddings, chosen for the no-GPU, no-API runtime constraint.

NumPy - vectorised similarity computation across candidate batches.

Pandas - submission validation and self-checks before output.

Python standard library (json, re, hashlib, datetime) - honeypot detection, filtering, and deterministic reasoning variation with zero added runtime dependencies.

Submission Assets

- GitHub Repository: [Github Link](#)
- Annotated Notebook: IntentRank_Candidate_Ranker.ipynb – 16-section layer-by-layer walkthrough
- Submission File: submission.csv – top 100 ranked candidates with evidence-grounded reasoning
- Sandbox / Demo: [Colab Link](#)
- Team Apex Climbers: Pratheeka B (Lead), Vidhya S, Mohammed Zaki S



INDIA.RUNS

Build what next India runs on

THANK YOU